

The PR Review Checklist

51 items across 8 categories. Every item includes a one-line "why" — a checklist without reasons just trains people to tick boxes without thinking.

This is the canonical, opinionated version. If your team wants to adapt it, start from [pr-review-checklist-template.md](#) instead and keep this one as the reference. If you'd rather work through this interactively with a progress tracker, use the [live checklist](#) or [interactive-checklist.html](#).

1. Context & Purpose

- ☐ **The PR description clearly states what changed and why.**
Why:
future readers (including you, in six months) need to understand intent without rejoining the issue tracker.
- ☐ **The PR links to the relevant issue, ticket, or spec.**
Why:
traceability matters more once a PR is old news and someone's asking "why did we do this?"
- ☐ **The PR's size matches its stated scope — no unrelated changes smuggled in.**
Why:
scope creep is the single biggest cause of slow, error-prone reviews.
- ☐ **The title is specific enough to be useful in a changelog.**
Why:
"fix bug" tells nobody anything a year from now; "Fix race condition in session token refresh" does.
- ☐ **You understand the problem this PR solves before reading a single line of code.**
Why:
you can't judge whether a solution is good without first knowing what it's solving for.

2. Architecture & Design

- ☐ **The change fits the existing architecture, or explicitly justifies deviating from it.**
Why:
silent architectural drift is how codebases end up with five different ways to do the same thing.
- ☐ **Responsibilities are placed in the right layer or module.**
Why:
business logic in a UI component or I/O in a "pure" function makes the code harder to test and reuse later.
- ☐ **The solution doesn't introduce a new pattern when an existing one already solves the problem.**
Why:
every new pattern is a new thing every future contributor has to learn.
- ☐ **Coupling is minimized — this change doesn't force unrelated modules to know about each other.**
Why:
tight coupling is what turns a five-minute fix into a four-file, three-team change next time.
- ☐ **The abstraction level matches the problem.**
Why:
an interface for a one-off script is over-engineering; a hardcoded value in something reused ten times is under-engineering.
- ☐ **Naming reflects domain concepts, not implementation details.**
Why:
`calculateShippingCost()` survives a database migration; `getRowFromOrdersTable()` doesn't.
- ☐ **The change is consistent with how similar problems were solved elsewhere in the codebase.**
Why:
consistency is what lets a reviewer (or new hire) recognize a pattern instead of re-deriving it.
- ☐ **If this introduces a new dependency, the trade-off is justified.**
Why:

bundle size, maintenance burden, and license terms are real costs, not afterthoughts.

3. Code Quality

- ☐ **Functions do one thing, and the name says what that thing is.**
Why:
a function that does three things can't be tested, reused, or understood in isolation.
- ☐ **No dead code, commented-out blocks, or leftover debug statements.**
Why:
version control already remembers deleted code — commented-out code is just noise with extra steps.
- ☐ **Variable and function names are descriptive without being verbose.**
Why:
userList beats both ul and theArrayThatContainsAllOfTheUsersInTheSystem.
- ☐ **Error handling is explicit — failures are caught and handled, not silently swallowed.**
Why:
an empty catch block is a bug report from production waiting to happen.
- ☐ **No magic numbers or strings without a named constant.**
Why:
if (status === 3) means nothing to the next person; if (status === ORDER_STATUS.SHIPPED) does.
- ☐ **Duplication is avoided, but not at the cost of readability.**
Why:
DRY is a guideline, not a religion — a forced abstraction to avoid three repeated lines can be worse than the repetition.
- ☐ **Control flow is easy to follow — no deep nesting that needs a flowchart to parse.**
Why:
if you need to count brace levels to know what's running, so will every future reader.
- ☐ **The diff is reviewable in scope and size.**
Why:
a 40-file PR doing "everything at once" is a process failure on the author's side, not a challenge for the reviewer to rise to.
- ☐ **Comments explain why, not what.**
Why:
the code already says what it does; a comment that just restates it in English adds maintenance cost with no benefit.
- ☐ **Edge cases — empty input, null, zero, max values — are visibly handled.**
Why:
the happy path is what gets demoed; the edge cases are what gets paged at 2 a.m.
- ☐ **The code doesn't rely on implicit behavior that breaks silently if a dependency changes.**
Why:
relying on undocumented ordering, default values, or side effects is a landmine for whoever upgrades that dependency.
- ☐ **Formatting and style match the project's conventions.**
Why:
if the linter didn't already catch this before the PR was opened, that's worth fixing in CI, not in every review.

4. Testing

- ☐ **New behavior has new tests — not just "existing tests still pass."**
Why:
passing tests prove you didn't break the old behavior; they say nothing about whether the new behavior works.
- ☐ **Tests verify behavior, not implementation details.**
Why:
tests that assert on internal structure break on every refactor, which trains people to stop refactoring.
- ☐ **At least one test covers the unhappy path, not just the golden path.**
Why:
the golden path is the 10% of cases that almost never causes an incident.

- ☐ **Tests are deterministic — no flaky timing, ordering, or unseeded randomness.**
Why:
a flaky test that gets re-run until it passes is worse than no test — it erodes trust in the whole suite.
- ☐ **If this fixes a bug, there's a regression test that would have caught it.**
Why:
otherwise the same bug reappears in eight months, in a slightly different shape.
- ☐ **Test names describe the scenario, so a failing test tells you what broke without opening the file.**
Why:
`test_1()` failing tells you nothing at 11 p.m.; `rejects_negative_quantity_on_checkout()` does.

5. Performance & Security

- ☐ **No obvious N+1 queries or unnecessary loops over large collections.**
Why:
this is invisible with ten rows of test data and catastrophic with ten million in production.
- ☐ **User input is validated and sanitized before it touches anything sensitive.**
Why:
"we'll validate it on the frontend" is not a security boundary.
- ☐ **Secrets, tokens, and credentials are not hardcoded or logged.**
Why:
once a secret hits version control history or a log aggregator, rotating it is the only real fix — deleting the line isn't enough.
- ☐ **New endpoints or data access respect existing authorization rules.**
Why:
a new feature that quietly bypasses an existing permission check is how horizontal privilege escalation bugs happen.
- ☐ **The change doesn't introduce a new attack surface without a clear mitigation.**
Why:
file uploads, deserialization of untrusted data, and raw SQL are where a huge share of real-world exploits start.
- ☐ **Memory and resource usage is bounded.**
Why:
an unbounded cache or a recursive function without a base case works fine in dev and takes down production under real load.
- ☐ **Third-party data — API responses, webhooks, uploads — is treated as untrusted.**
Why:
"it's from our own partner's API" is not the same as "it's safe to trust without validation."
- ☐ **If this touches a hot path, there's evidence performance was actually considered.**
Why:
a benchmark, a profiling note, or even a sentence of reasoning beats "it felt fine on my machine."

6. Documentation

- ☐ **Public functions, classes, and endpoints have docs where the "why" isn't obvious from the name.**
Why:
good naming reduces the need for comments; it doesn't eliminate it.
- ☐ **README, API docs, or onboarding docs are updated if this changes how the project is used.**
Why:
outdated docs are worse than no docs — they actively mislead the next person.
- ☐ **Breaking changes are called out explicitly, not buried in the diff.**
Why:
nobody reads every line of every PR; a breaking change deserves a flag, not an Easter egg.
- ☐ **Configuration or environment variable changes are documented.**
Why:
a new required env var with no documentation is a guaranteed support ticket from the next person who pulls main.
- ☐ **If this adds a new dependency or tool, there's a one-line note on why.**
Why:

saves the next person from reverse-engineering the decision from a commit hash.

7. Standards Compliance

- ☐ **The change follows the team's style guide and architectural decision records, if any exist.**
Why:
a decision the team already made shouldn't get re-litigated in every PR comment thread.
- ☐ **CI checks pass — and you actually read why, not just that the badge is green.**
Why:
a flaky CI pass that happened to go green this time isn't the same as the code being correct.
- ☐ **Commit messages follow the project's convention and are useful on their own, out of context.**
Why:
`git blame` is most useful exactly when you have zero other context — that's the moment the commit message has to carry the weight.
- ☐ **Accessibility basics are respected for any user-facing change.**
Why:
labels, contrast, and keyboard navigation aren't "nice to have" — they're the difference between a feature working for everyone or only some users.

8. Final Considerations

- ☐ **You'd be comfortable being paged about this code at 3 a.m.**
Why:
this is the single best gut-check for "do I actually understand what I just approved."
- ☐ **The feedback you left is about the code, not the person who wrote it.**
Why:
"this function is doing too much" invites a conversation; "you always overcomplicate things" invites a defense.
- ☐ **If you didn't fully review something — a generated file, a vendored library — you said so instead of silently approving it.**
Why:
an approval should mean what it says; silently rubber-stamping what you skimmed defeats the entire point of review.

Want this without having to scroll a markdown file? Use the [interactive version](#) — same 51 items, a progress bar, and a "Review Readiness" score per category.